

Rapport DevOps - Mini-projet Helpdesk API

Nom de l'établissement: a compléter

Module: DevOps | Année universitaire: 2025-2026

Titre: Plateforme de tickets de support (mini Helpdesk)

Étudiants: Mouaad Chakir et Souad Chakir

Enseignant: Pr. Soufiane HAMIDA

Dépôt Git: <https://github.com/mouaadchakir/devops-helpdesk-api>

1. Introduction

Le projet consiste a developper une API REST de gestion de tickets support et a l'integrer dans une demarche DevOps complete.

- API REST documentee et securisee
- Workflow Git avec branches et PR
- Conteneurisation Docker
- Pipeline CI/CD automatise
- Bonus Kubernetes

2. Architecture de l'application

L'architecture suit une separation en couches: controllers, services, repositories et persistance MySQL.

Les roles USER/AGENT/ADMIN gouvernent les droits d'accès.

3. Mise en oeuvre

Structure du code

Organisation en packages métier pour limiter le couplage.

Modele de donnees

- AppUser, Role, Ticket, TicketResponse

Fonctionnalites

- Creation/consultation tickets
- Assignment, statut, reponses

4. Strategie Git

Workflow main/develop/feature avec Pull Requests et resolution des conflits avant merge.

5. Conteneurisation

Dockerfile multi-stage et docker-compose pour API + MySQL en local.

6. Pipeline CI/CD

GitHub Actions execute tests/build et publie l'image sur GHCR pour main.

Lien: <https://github.com/mouaadchakir/devops-helpdesk-api/actions/workflows/ci-cd.yml>

7. Kubernetes (bonus)

Deploiement via kubectl apply -k k8s/ et validation par etat pods/services + acces applicatif.

8. Difficultes et solutions

- Conflit port MySQL
- Erreurs 403 sur roles
- Activation contexte Kubernetes
- Acces NodePort via port-forward

9. Repartition du travail

Mouaad Chakir

- Backend, securite, Docker/CI

Souad Chakir

- Documentation, captures, validation finale

10. Conclusion et perspectives

Le projet satisfait les exigences DevOps demandees.

Perspectives: JWT, tests integration avances, observabilite, cloud.

Figures et analyses

Figure 1 - Architecture globale

01_architecture.png

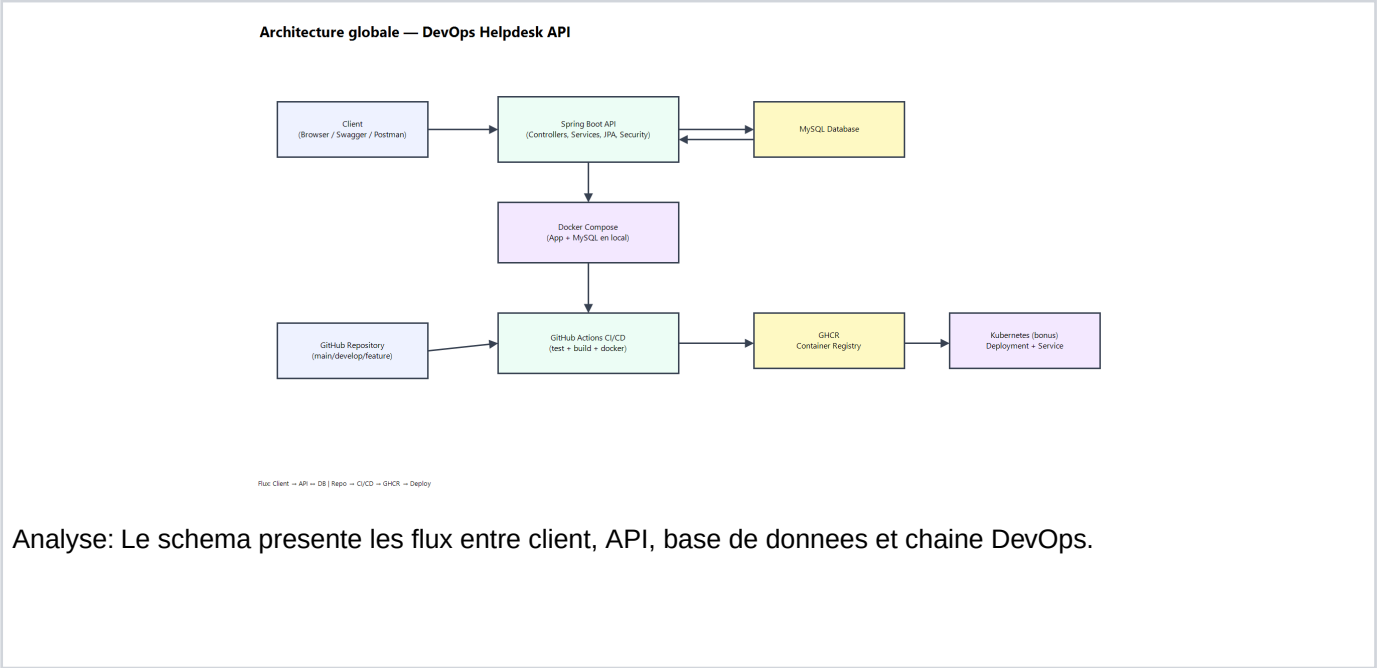
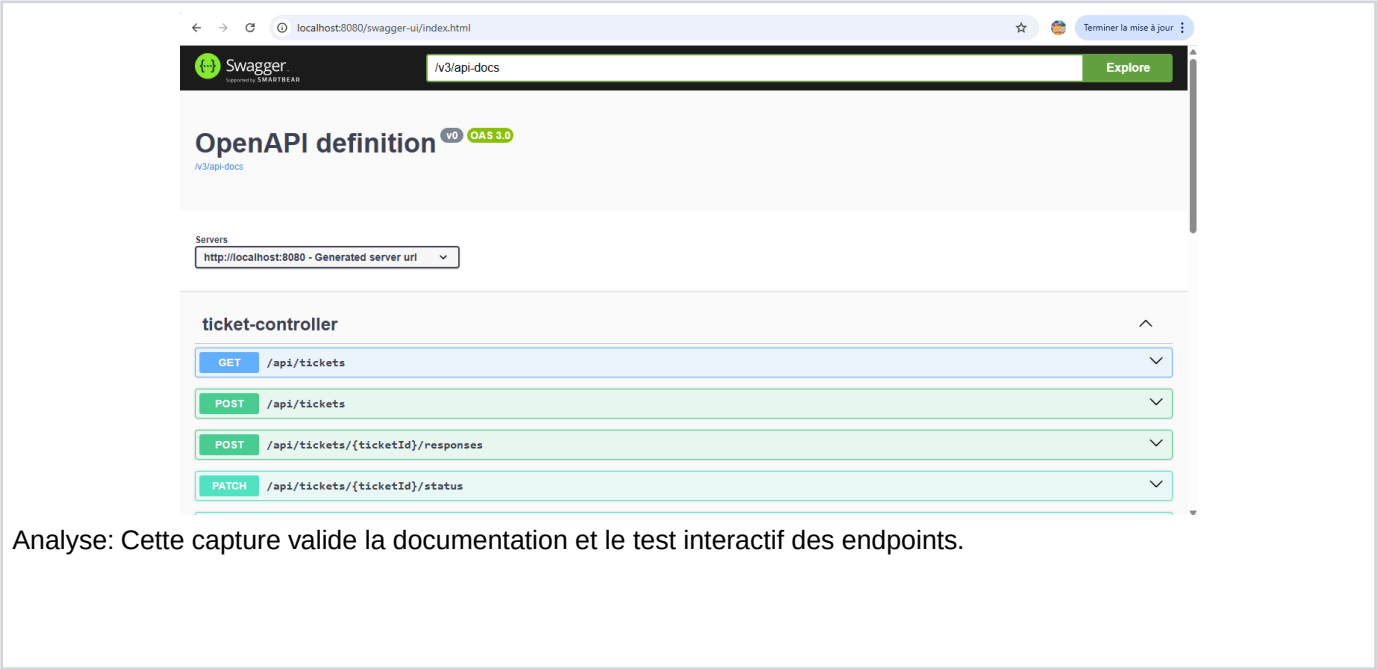


Figure 2 - Swagger UI

02_swagger_home.png



Figures et analyses

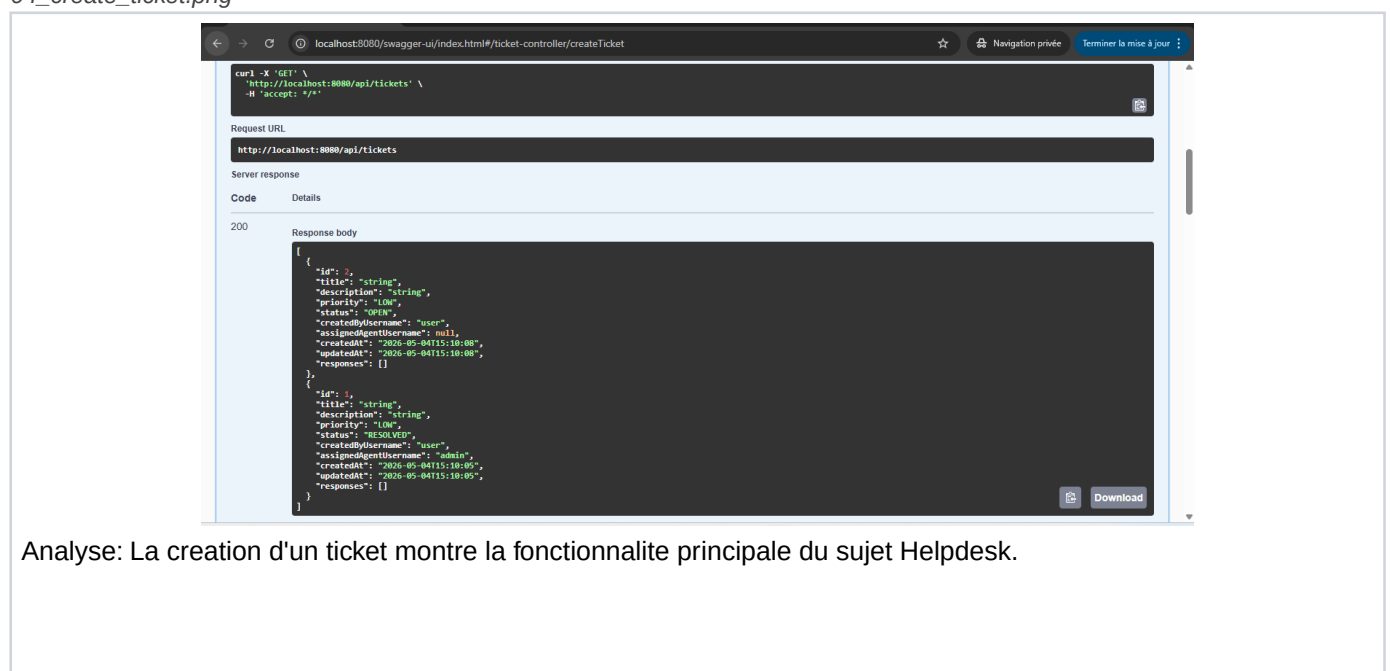
Figure 3 - Authentification

03_auth_me_success.png



Figure 4 - Creation ticket

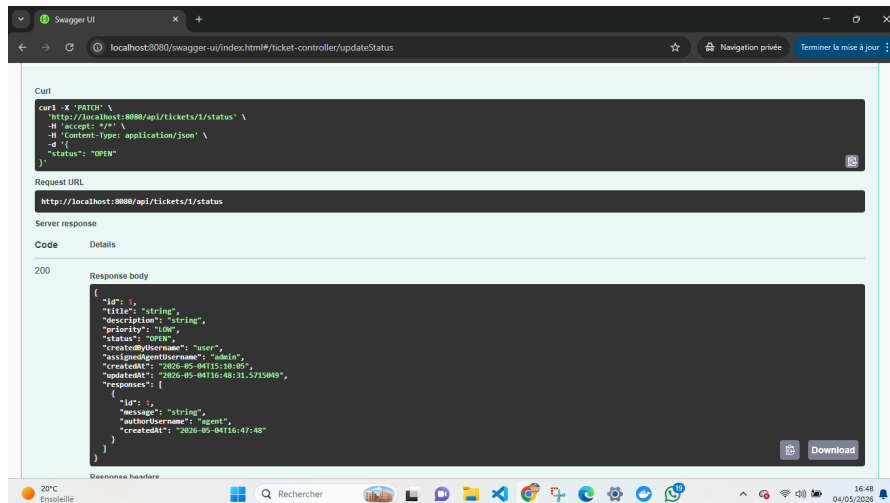
04_create_ticket.png



Figures et analyses

Figure 5 - Assignment / statut

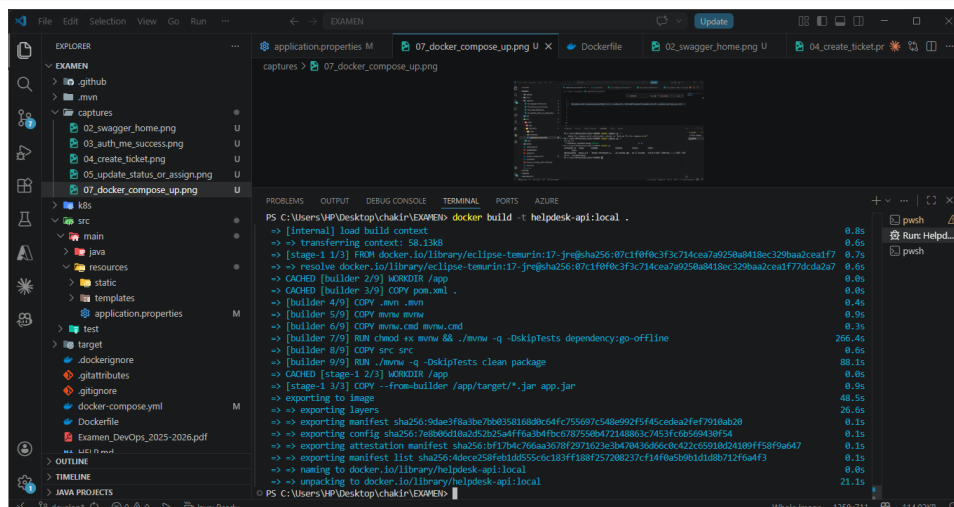
05_update_status_or_assign.png



Analyse: Cette etape valide le cycle de vie du ticket et les droits AGENT/ADMIN.

Figure 6 - Build Docker

06_docker_build.png

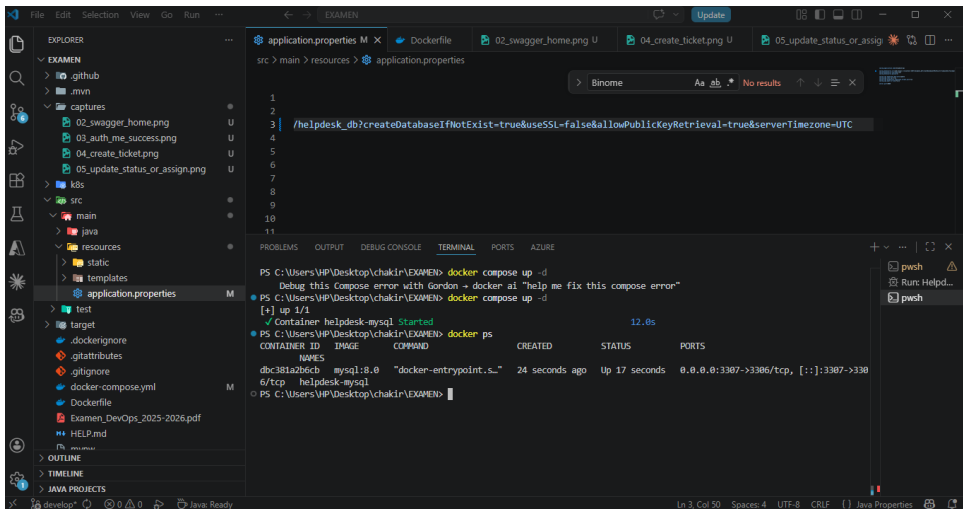


Analyse: Preuve de packaging reproductible de l'application en image conteneurisee.

Figures et analyses

Figure 7 - Docker Compose

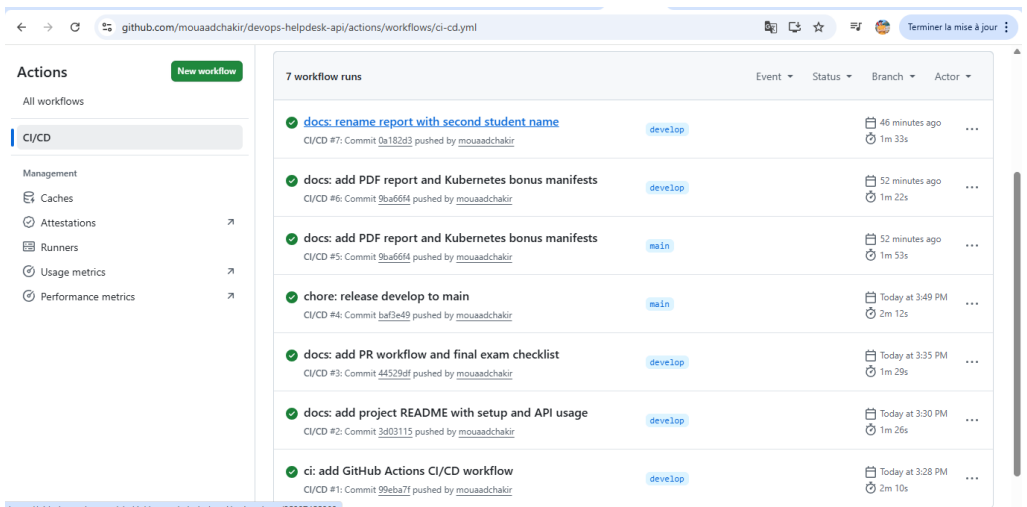
07_docker_compose_up.png



Analyse: La stack locale API + MySQL démarre correctement avec docker compose.

Figure 8 - Pipeline CI/CD vert

08_ci_pipeline_green.png

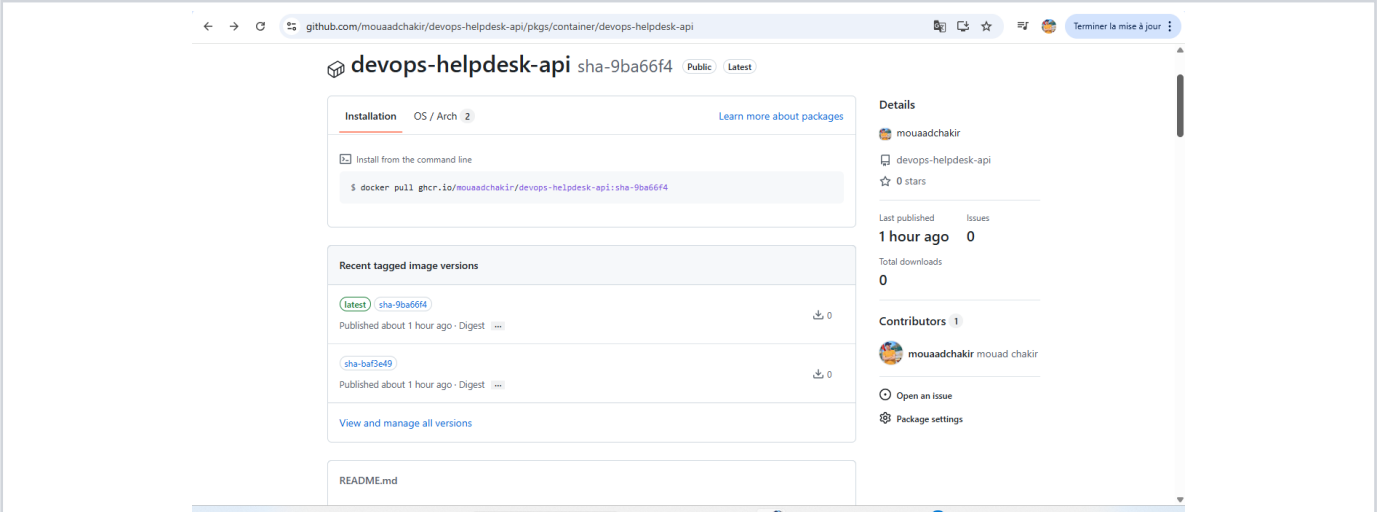


Analyse: La chaîne d'intégration passe au vert après tests et build.

Figures et analyses

Figure 9 - Publication GHCR

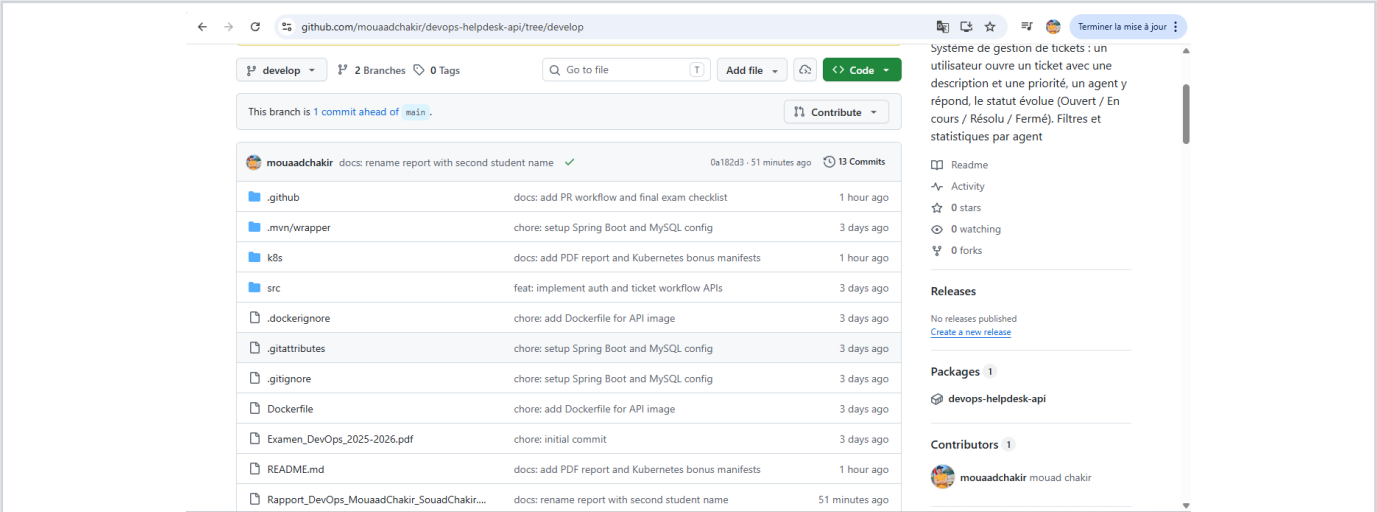
09_ghcr_image.png



Analyse: L'image est bien publiee sur le registre conteneur GitHub.

Figure 10 - Workflow Git

10_git_pr_or_branches.png

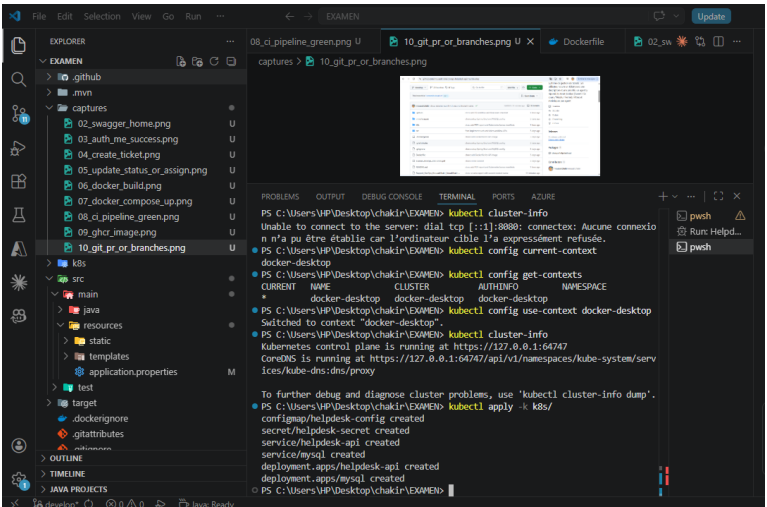


Analyse: Les branches et PR prouvent une collaboration structuree.

Figures et analyses

Figure 11 - Kubectl apply

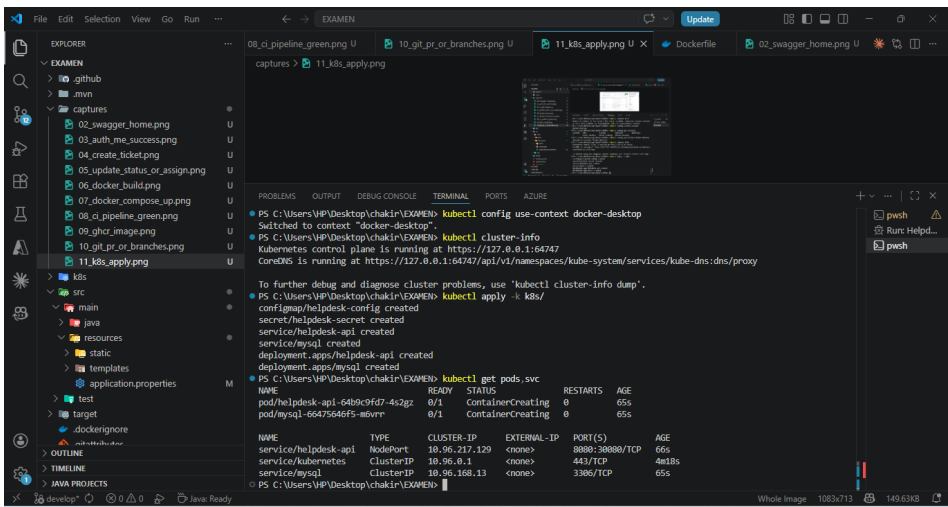
11_k8s_apply.png



Analyse: Les manifestes Kubernetes sont appliqués sur le cluster.

Figure 12 - Pods et services

12_k8s_get_pods_svc.png

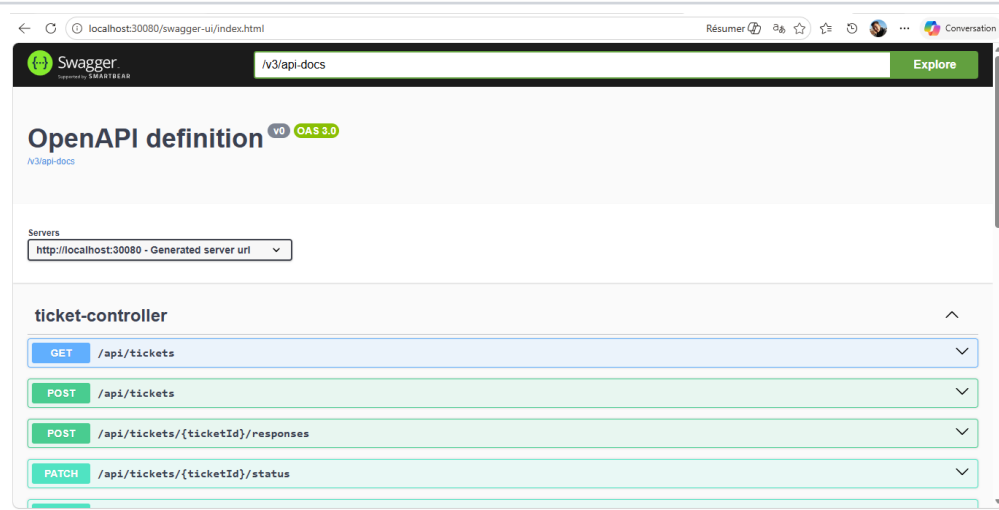


Analyse: Les ressources K8s sont en état opérationnel.

Figures et analyses

Figure 13 - Swagger via Kubernetes

13_k8s_swagger_access.png



Analyse: La disponibilité applicative est vérifiée après déploiement.